

Comprehensive Research Report: Advanced Trading Strategies for Prediction Markets

Executive Summary

This report synthesizes cutting-edge research on algorithmic trading strategies applicable to prediction markets like Polymarket. It provides detailed implementation guidance for four interconnected approaches: (1) CPPI/TIPP-MADDPG portfolio insurance with multi-agent reinforcement learning, (2) FinAgent-style LLM alpha mining, (3) Kalshi volatility collapse trading, and (4) regime-switching frameworks. Each strategy is explained with mathematical formulations, implementation steps, and practical considerations for production deployment.

Introduction

Prediction markets represent a frontier in financial technology, combining elements of traditional options markets, behavioral economics, and real-time information aggregation. The strategies outlined in this report leverage recent advances in machine learning, reinforcement learning, and large language models to exploit structural inefficiencies in these markets while managing risk through sophisticated portfolio insurance mechanisms.

The research base includes three seminal papers:

1. Zhang et al. (2023) - CPPI/TIPP-MADDPG for quantitative trading[1]
2. Diercks, Katz, & Wright (2026) - Kalshi macroeconomic prediction market analysis[2]
3. Zhang et al. (2024) - FinAgent multimodal trading agent[3]

1. CPPI-MADDPG Strategy: Portfolio Insurance with Multi-Agent Reinforcement Learning

1.1 Conceptual Foundation

Constant Proportion Portfolio Insurance (CPPI) is a dynamic hedging strategy that automatically adjusts exposure between risky and risk-free assets to maintain a minimum guaranteed portfolio value (the "floor") [4][5].

Core Mechanism:

Total Asset (A) = Protection Floor (F) + Cushion (C)

Risky Asset Exposure (E) = $k \times C = k \times (A - F)$

Where:

- **F (Floor):** Minimum guaranteed portfolio value
- **C (Cushion):** Amount you can afford to lose ($A - F$)
- **k (Multiplier):** Risk appetite parameter (typically 3-6)
- **E (Exposure):** Capital allocated to risky trading

Example: With \$10,000 portfolio, 80% floor (\$8,000), and $k=4$:

- Cushion = $\$10,000 - \$8,000 = \$2,000$
- Risky exposure = $4 \times \$2,000 = \$8,000$
- If portfolio grows to \$11,000: New exposure = $4 \times (\$11,000 - \$8,000) = \$12,000$
- If portfolio drops to \$9,000: New exposure = $4 \times (\$9,000 - \$8,000) = \$4,000$

1.2 Time-Invariant Portfolio Protection (TIPP)

TIPP improves on CPPI by making the floor **dynamic** rather than fixed [6]:

$$F_t = \max\{\phi \times A_t, F_{t-1}\}$$

$$E_t = k \times (A_t - F_t)$$

Where:

- **ϕ (Floor percentage):** Typically 0.85-0.95
- **F_t :** Floor "ratchets up" as portfolio grows, locking in gains
- Never decreases even if portfolio falls

Key Advantage: Eliminates "gap risk" - the danger that a rapid market crash breaches the floor before you can rebalance[1].

Performance Comparison (Zhang et al. 2023):

Strategy	Annual Return	Sharpe Ratio	Max Drawdown
MADDPG	8.22%	1.99	12.26%
CPPI-MADDPG	7.76%	2.18	6.60%
TIPP-MADDPG	9.68%	2.09	9.02%

1.3 Multi-Agent Deep Deterministic Policy Gradient (MADDPG)

MADDPG trains multiple AI agents to collaboratively learn optimal trading policies in continuous action spaces[7].

Mathematical Framework:

Stochastic Game Definition:

$\langle N, S, \{A_i\}, P, \{R_i\}, \gamma \rangle$

- **N:** Number of agents
- **S:** Shared state space (market conditions)
- **A_i :** Action space for agent i (position weights)
- **P:** State transition dynamics $S \times A \rightarrow \Delta(S)$
- **R_i :** Reward function for agent i
- **γ :** Discount factor (0.95-0.99)

Agent Architecture:

Each agent has:

1. **Actor Network π_θ :** Maps states to actions (trading decisions)

2. **Critic Network** Q_φ : Evaluates action quality given all agents' actions

Policy Gradient Update:

$$\nabla_{\theta_i} J(\theta_i) = E_{\{s \sim D\}} [\nabla_{\theta_i} \log \pi_{\theta_i}(a_i | s) \cdot \nabla_{a_i} Q_\varphi(s, a_1, \dots, a_N) |_{a_i = \pi_{\theta_i}(s)}]$$

Critic Loss Function (with diversity constraint):

$$L(\varphi_i) = \lambda \cdot E[(Q_\varphi(s, a_1, \dots, a_N) - y)^2] + (1 - \lambda) \cdot \sum_{j \neq i} \text{Corr}(a_i, a_j)^2$$

$$y = r_i + \gamma \cdot Q_\varphi(s', a'_1, \dots, a'_N)$$

The **correlation penalty** ensures agents pursue diverse strategies rather than converging to identical behavior[1].

1.4 Integration: CPPI/TIPP Layer Over MADDPG

The Innovation: Use MADDPG to generate raw trading signals, then apply CPPI/TIPP constraints to enforce risk management[1].

Algorithm Workflow:

Pseudocode for TIPP-MADDPG

```
def tipp_maddpg_trading(initial_capital, floor_pct, multiplier,
num_agents):
    # Initialize
    portfolio_value = initial_capital
    floor = floor_pct * initial_capital
    agents = [MADDPGAgent(i) for i in range(num_agents)]
```

```
    for trading_day in trading_period:
        # 1. Observe market state
        state = get_market_state() # prices, volume, technical indicators

        # 2. Each agent proposes action (position weights)
        raw_actions = [agent.select_action(state) for agent in agents]

        # 3. Apply TIPP constraint
        cushion = portfolio_value - floor
```

```

max_risky_exposure = multiplier * cushion

# Scale actions to respect TIPP limit
constrained_actions = []
for action in raw_actions:
    risky_portion = sum(abs(action)) # total position size
    if risky_portion > max_risky_exposure:
        # Scale down proportionally
        scale_factor = max_risky_exposure / risky_portion
        action = action * scale_factor
    constrained_actions.append(action)

# 4. Execute trades
execute_trades(constrained_actions)

# 5. Observe results
new_portfolio_value, rewards = observe_outcomes()

# 6. Update floor (TIPP ratchet mechanism)
floor = max(floor_pct * new_portfolio_value, floor)

# 7. Store experience and train
for i, agent in enumerate(agents):
    agent.store_transition(state, constrained_actions[i],
                          rewards[i], new_state)
    agent.train() # Update actor and critic networks

portfolio_value = new_portfolio_value

```

Key Implementation Details:

1. State Representation:

```

state = {
    'prices': [p_1, p_2, ..., p_D], # D assets
    'holdings': [h_1, h_2, ..., h_D],
    'balance': cash_balance,
    'technical_indicators': {
        'rsi': [...],

```

```

'macd': [...],
'bollinger': [...]
},
'market_sentiment': sentiment_score
}

```

2. Action Space:

Each agent outputs a vector $a \in [-1, 1]^D$:

- Positive: Buy signal strength
- Negative: Sell signal strength
- Magnitude: Position size

3. Reward Function:

$r_t = (\text{PortfolioValue}_t - \text{PortfolioValue}_{t-1}) - \text{transaction_costs}$

4. Network Architecture:

- **Actor:** 3-layer MLP (256-128-64 units, ReLU activation, tanh output)
- **Critic:** 4-layer MLP (512-256-128-1 units, ReLU activation, linear output)
- **Optimizer:** Adam (learning rate $1e-4$ for actor, $1e-3$ for critic)

5. Training Hyperparameters:

```

config = {
'num_agents': 5,
'replay_buffer_size': 100000,
'batch_size': 256,
'gamma': 0.99, # discount factor
'tau': 0.01, # soft update rate for target networks
'lambda': 0.7, # balance between Q-loss and diversity
'floor_pct': 0.85, # TIPP floor percentage
'multiplier': 4 # risk multiplier
}

```

1.5 Implementation for Polymarket

Adaptation Strategy:

Market Structure Differences:

- Polymarket contracts are binary options (0 or 1 payoff)
- Prices represent probabilities (0.01 to 0.99)
- No continuous price movement - discrete outcome resolution

Modified State Space:

```
polymarket_state = {
'contract_prices': [p_yes_1, p_yes_2, ..., p_yes_N],
'implied_probabilities': [prob_1, prob_2, ..., prob_N],
'volume_24h': [vol_1, vol_2, ..., vol_N],
'time_to_resolution': days_remaining,
'related_news_sentiment': sentiment_scores,
'order_book_depth': bid_ask_spreads,
'kalshi_correlations': cross_market_signals # exploit arbitrage
}
```

Action Space Modification:

For each contract i:

```
action[i] = {
'direction': 'buy_yes' | 'buy_no' | 'sell_yes' | 'sell_no' | 'hold',
'quantity': position_size, # constrained by TIPP
'limit_price': price_threshold # optional
}
```

TIPP Application:

Calculate maximum position size across all contracts

```
total_exposure = sum(contract.quantity * contract.price for contract
in positions)
cushion = portfolio_value - (0.85 * portfolio_value)
max_exposure = 4 * cushion
```

```
if total_exposure > max_exposure:
# Scale down all positions proportionally
scale = max_exposure / total_exposure
```

for contract in positions:
contract.quantity *= scale

Key Considerations:

1. **Binary outcomes require modified reward:** Use probability-weighted expected value
2. **Time decay:** Incorporate time-to-resolution in state representation
3. **Liquidity constraints:** Check volume before sizing positions
4. **Correlation exploitation:** Use multiple agents to trade correlated events

2. LLM Alpha Miner: FinAgent-Style Signal Generation

2.1 Conceptual Architecture

FinAgent represents a paradigm shift: using **Large Language Models (LLMs)** to automatically discover trading signals by processing multimodal data (text, numbers, images)[3].

Five Core Modules:

1. **Market Intelligence Module:** Processes news, prices, charts
2. **Memory Module:** Stores and retrieves historical patterns
3. **Low-Level Reflection:** Correlates news → price movements
4. **High-Level Reflection:** Evaluates past trading decisions
5. **Decision-Making Module:** Generates final trading signals

2.2 Market Intelligence Module

Purpose: Extract actionable insights from diverse data sources.

Data Sources:

- **Textual:** News articles, social media, earnings transcripts, forum discussions
- **Numerical:** Price data, volume, order flow, technical indicators
- **Visual:** Candlestick charts, heatmaps, correlation matrices

Processing Workflow:


```

def market_intelligence(current_date, asset):
# 1. Gather latest data
latest_news = fetch_news(asset, date=current_date, lookback=1)
latest_prices = fetch_prices(asset, date=current_date, lookback=30)
kline_chart = generate_chart(latest_prices)

# 2. Sentiment analysis (LLM-based)
prompt = f"""
Analyze the following news for {asset}:
{latest_news}

For each item, determine:
1. Sentiment: POSITIVE, NEGATIVE, or NEUTRAL
2. Time horizon impact: SHORT-TERM (<1 week), MEDIUM-TERM (1-4 weeks)
3. Price impact magnitude: LOW, MEDIUM, HIGH

Provide summary and investment implications.
"""

latest_summary = llm_call(prompt, model='gpt-4')

# 3. Diversified retrieval - generate query texts
query_texts = llm_call(f"""
Based on summary: {latest_summary}

Generate retrieval queries for:
- short_term_query: Events with <1 week impact
- medium_term_query: Events with 1-4 week impact
- long_term_query: Events with >4 week impact
- bullish_query: Positive price drivers
- bearish_query: Negative price drivers
""", model='gpt-4')

# 4. Retrieve similar historical intelligence
past_intelligence = []
for query_type, query_text in query_texts.items():
    embeddings = get_embeddings(query_text, model='text-embedding-3-large')
    similar_past = vector_db.similarity_search(

```

```

        embeddings,
        top_k=5,
        filter={'query_type': query_type}
    )
    past_intelligence.extend(similar_past)

# 5. Summarize historical patterns
historical_summary = llm_call(f"""
Past similar situations:
{past_intelligence}

What patterns emerged? How did prices react?
Extract key lessons for current situation.
""", model='gpt-4')

return {
    'latest_summary': latest_summary,
    'historical_summary': historical_summary,
    'query_texts': query_texts,
    'chart': kline_chart
}

```

Diversified Retrieval Innovation:

Traditional approach: Use summary as query → noisy, unfocused retrieval

FinAgent approach: Generate **separate query texts** for different retrieval types[3]:

- Short-term vs medium-term vs long-term impacts
- Bullish vs bearish signals
- News vs reports vs earnings
- Price increase vs decrease contexts

Benefit: Retrieve $M \times K$ relevant items (M retrieval types $\times$ K top results each) with precision.

2.3 Low-Level Reflection Module

Purpose: Learn correlation between market intelligence and price movements.

Visual Input: Kline (candlestick) chart with technical indicators (MACD, RSI, Bollinger Bands)

Implementation:

```
def low_level_reflection(market_intel, price_data, kline_chart_image):  
    # Calculate price movements  
    short_term_change = (price_data[-1] - price_data[-1]) / price_data[-1] #  
    1 day  
    medium_term_change = (price_data[-1] - price_data[-7]) /  
    price_data[-7] # 7 days  
    long_term_change = (price_data[-1] - price_data[-30]) / price_data[-30]  
    # 30 days
```

```
# LLM analysis with vision (GPT-4V)  
prompt = f"""  
You are analyzing price movements for trading decisions.
```

```
Market Intelligence Summary:
```

```
{market_intel['latest_summary']}
```

```
{market_intel['historical_summary']}
```

```
Price Movements:
```

```
- Short-term (1 day): {short_term_change:.2%}
```

```
- Medium-term (7 days): {medium_term_change:.2%}
```

```
- Long-term (30 days): {long_term_change:.2%}
```

```
Kline Chart (see image with MACD, RSI, Bollinger Bands)
```

```
Analyze:
```

1. How did the market intelligence lead to these price movements?
2. Which factors (news, technical indicators) were most influential?
3. Are there divergences between fundamentals and technicals?

Provide:

- reasoning: Detailed analysis
 - query: Summary for future retrieval
- """

```
reflection = llm_call_with_vision(
    prompt,
    images=[kline_chart_image],
    model='gpt-4-vision-preview'
)

# Store in memory for future retrieval
embeddings = get_embeddings(reflection['query'])
memory_db.store({
    'type': 'low_level_reflection',
    'date': current_date,
    'content': reflection,
    'embeddings': embeddings
})

return reflection
```

Key Insight: By analyzing Kline charts visually, the LLM can detect patterns invisible to pure numerical analysis (e.g., head-and-shoulders formations, support/resistance levels, volume divergences).

2.4 High-Level Reflection Module

Purpose: Learn from past trading decisions to avoid repeating mistakes.

Visual Input: Trading chart showing buy/sell points and cumulative returns

Implementation:

```
def high_level_reflection(market_intel, low_level_reasoning,
    trading_history, trading_chart_image):
```

```
# Get past 14 days of decisions
recent_decisions = trading_history[-14:]
```

```
prompt = f"""
You are evaluating your past trading performance.
```

```

Past 14 Days Decisions:
{format_decisions(recent_decisions)}
```

```

Market Intelligence:
{market_intel['latest_summary']}
```

```

Price Movement Analysis:
{low_level_reasoning}
```

```

Trading Chart (see image showing buy/sell points and returns)
```

```

For each decision:
```

1. Was it correct? Why or why not?
2. If wrong, how should it have been different?
3. What lessons can improve future decisions?

```

Provide:
```

- reasoning: Analysis of each decision
- improvement: Specific corrections for mistakes
- summary: Key lessons for future
- query: Summary for retrieval

```
"""
```

```
reflection = llm_call_with_vision(
    prompt,
    images=[trading_chart_image],
    model='gpt-4-vision-preview'
)
```

```
# Store in memory
```

```
embeddings = get_embeddings(reflection['query'])
```

```
memory_db.store({
    'type': 'high_level_reflection',
    'date': current_date,
    'content': reflection,
    'embeddings': embeddings
})

return reflection
```

Learning Mechanism:

1. **Identify mistakes:** "The BUY on 2023-06-01 was premature - RSI was overbought"
2. **Suggest corrections:** "Should have waited for RSI < 50 confirmation"
3. **Extract principles:** "Don't chase momentum without technical confirmation"
4. **Apply to future:** Retrieve similar scenarios and apply learned lessons

2.5 Decision-Making Module with Tool Augmentation

Purpose: Integrate all inputs and domain knowledge to make final trading decision.

Augmented Tools:

1. **Expert Guidance:** Commentary from professional analysts
2. **MACD Crossover:** Momentum indicator (signal line crosses MACD line)
3. **KDJ & RSI Filter:** Oscillator for overbought/oversold + confirmation
4. **Z-Score Mean Reversion:** Statistical arbitrage signal

Implementation:

```
def make_trading_decision(market_intel, low_level_reflection,
    high_level_reflection, augmented_tools,
    trader_preference='moderate'):
    # Retrieve relevant past reflections
    past_low_level = retrieve_from_memory('low_level_reflection',
```

```
query=low_level_reflection['query'],
top_k=3)
past_high_level = retrieve_from_memory('high_level_reflection',
query=high_level_reflection['query'],
top_k=3)
```

```
# Calculate traditional indicators
macd_signal = augmented_tools.macd_crossover() # 'BUY', 'SELL', 'HOLD'
kdj_rsi_signal = augmented_tools.kdj_rsi_filter()
mean_reversion_signal = augmented_tools.z_score_reversion()
```

```
# Get expert guidance (if available)
expert_analysis = augmented_tools.fetch_expert_guidance()
```

```
# Comprehensive decision prompt
prompt = f"""
You are making a trading decision. Analyze step-by-step:
```

1. MARKET INTELLIGENCE:

```
Latest: {market_intel['latest_summary']}
Historical: {market_intel['historical_summary']}
Sentiment: [Extract from above]
```

2. PRICE MOVEMENT ANALYSIS:

```
{low_level_reflection['reasoning']}
Similar past patterns: {past_low_level}
```

3. PAST DECISION REFLECTIONS:

```
{high_level_reflection['summary']}
Similar past lessons: {past_high_level}
```

4. TRADITIONAL INDICATORS:

```
- MACD: {macd_signal}
- KDJ & RSI: {kdj_rsi_signal}
- Mean Reversion: {mean_reversion_signal}
```

5. EXPERT GUIDANCE:

```
{expert_analysis}
```

6. TRADER PREFERENCE:

Risk tolerance: {trader_preference}

Provide:

- analysis: How each factor influences decision
- reasoning: Detailed justification
- action: Exactly one of [BUY, SELL, HOLD]
- confidence: 0-100%

"""

decision = llm_call(prompt, model='gpt-4')

return decision

Chain-of-Thought Reasoning Example:

ANALYSIS:

1. Market Intelligence shows POSITIVE sentiment (AR/VR product excitement)
2. Price movement analysis indicates bullish trend (1.6% 1-day, 3.38% 7-day gains)
3. Past reflections show similar product launches drove 15%+ rallies
4. MACD shows golden cross (bullish), RSI at 58 (not overbought)
5. Experts are upgrading price targets

REASONING:

Despite NSA concerns (minor negative), the dominant signal is positive:

- Product cycle tailwinds align with historical patterns
- Technical indicators confirm uptrend
- Risk/reward favorable with RSI room to run
- Expert consensus bullish

ACTION: BUY

CONFIDENCE: 78%

2.6 Performance Results

FinAgent vs Baselines (Zhang et al. 2024):

Method	Annualized Return	Sharpe Ratio	Max Drawdown
Buy & Hold	12.4%	0.89	-28.3%
MACD	8.7%	0.72	-19.5%
LSTM	15.2%	1.21	-22.1%
PPO (RL)	18.9%	1.45	-18.7%
FinGPT	24.3%	1.68	-15.2%
FinAgent	32.6%	1.94	-12.8%

Best single result: **92.27% return** (84.39% relative improvement over best baseline)[3]

2.7 Implementation for Polymarket

Adaptations:

1. Market Intelligence Sources:

```
polymarket_data_sources = {  
'contract_descriptions': polymarket_api.get_event_description(),  
'related_news': newsapi.search(event_keywords),  
'social_sentiment': twitter_api.search(event_hashtags),  
'related_markets': fetch_correlated_contracts(), # e.g., Fed rate ↔  
inflation  
'expert_predictions': aggregator.fetch_forecasts(), # FiveThirtyEight,  
etc.  
'historical_similar_events': vector_db.search_past_events()  
}
```

2. Visual Data:

- Price history chart for contract (Yes/No price over time)
- Volume profile

- Order book depth visualization
- Correlation heatmap with related contracts

3. Reflection Specific to Binary Outcomes:

Low-level: Why did Yes price move from 0.45 → 0.62?

low_level_prompt = f"""

The contract '{contract_name}' Yes price moved from 45¢ to 62¢ in 3 days.

News during period:
{relevant_news}

What drove this price movement? Analyze:

1. Which news items were most impactful?
2. Was the move justified by fundamentals?
3. Is there momentum or is it overdone?

"""

High-level: Was buying Yes at 45¢ a good decision?

high_level_prompt = f"""

You bought Yes at 45¢, now at 62¢ (3 days later).

Reflection:

1. Was entry timing optimal? (Could have waited?)
2. Should you take profit now or hold until resolution?
3. What would you do differently?

"""

4. Tool Augmentation:

Traditional indicators don't directly apply (no continuous price), so adapt:

```
class PolymarketTools:
    def momentum_signal(self, price_history):
        """Detect if price is trending"""
        recent_slope = np.polyfit(range(len(price_history[-7:])),
            price_history[-7:], 1)[0]
        return 'BUY_YES' if recent_slope > 0.02 else 'BUY_NO' if recent_slope <
            -0.02 else 'HOLD'
```

```
    def mean_reversion_signal(self, price_history):
        """Check if price has deviated from historical mean"""
        mean_price = np.mean(price_history)
        current_price = price_history[-1]
        z_score = (current_price - mean_price) / np.std(price_history)

        if z_score > 2: # Overpriced
            return 'BUY_NO' # Bet against current outcome
        elif z_score < -2: # Underpriced
            return 'BUY_YES'
        return 'HOLD'

    def expert_consensus(self, event_id):
        """Aggregate external forecasts"""
        forecasts = fetch_expert_forecasts(event_id)
        consensus_prob = np.mean([f.probability for f in forecasts])
        market_price = get_current_price(event_id)

        if consensus_prob > market_price + 0.05:
            return 'BUY_YES' # Experts more bullish than market
        elif consensus_prob < market_price - 0.05:
            return 'BUY_NO'
        return 'HOLD'
```

3. Kalshi-Implied Volatility Trading

3.1 Kalshi as Macro Features

Core Insight: Kalshi prediction markets provide real-time probability distributions for macro events (CPI, Fed rates, unemployment) that are **unavailable** from traditional sources[2].

What Kalshi Offers:

- **Full distributions** (not just point forecasts): Mean, variance, skewness, kurtosis
- **High-frequency updates** (intraday, event-driven)
- **Macro variables without option markets:** GDP, core CPI, unemployment, Fed funds rate at specific FOMC meetings

Use as Features:

```
def extract_kalshi_features(event_date):
```

```
    """Extract distributional features from Kalshi markets"""
```

```
    # Fetch Kalshi distribution for CPI
```

```
    cpi_distribution = kalshi_api.get_distribution('CPI_MOM', event_date)
```

```
    features = {
```

```
        # First moment
```

```
        'cpi_mean': np.average(cpi_distribution.outcomes,  
                               weights=cpi_distribution.probabilities),
```

```
        # Second moment
```

```
        'cpi_variance': np.average((cpi_distribution.outcomes - cpi_mean)**2,  
                                   weights=cpi_distribution.probabilities),
```

```
        'cpi_volatility': np.sqrt(cpi_variance),
```

```
        # Third moment (asymmetry)
```

```
        'cpi_skewness': calculate_skewness(cpi_distribution),
```

```
        # Tail risk
```

```
        'cpi_prob_above_3pct': sum(cpi_distribution.probabilities[cpi_distribution.outcomes > cpi_mean + 3 * cpi_volatility])
```

```

'cpi_prob_below_2pct': sum(cpi_distribution.probabilities[cpi_distribution.

# Cross-market features
'fed_rate_mean': kalshi_api.get_distribution('FED_RATE', next_fomc).mean,
'fed_rate_variance': kalshi_api.get_distribution('FED_RATE', next_fomc).var

# Correlation
'cpi_fed_correlation': calculate_correlation(cpi_distribution, fed_rate_distri
}

return features

```

Integration with Polymarket:

Polymarket trades often depend on macro conditions. Example correlations:

- **"Will recession occur in 2026?"** ↔ Kalshi unemployment/GDP distributions
- **"Will Fed cut rates by June?"** ↔ Kalshi Fed funds rate distribution
- **"Will inflation exceed 3%?"** ↔ Kalshi CPI distribution

Trading Strategy:

```

def polymarket_trade_with_kalshi_features(polymarket_contract):
# Get Kalshi macro features
kalshi_features =
extract_kalshi_features(polymarket_contract.resolution_date)

```

```

# Extract relevant correlation
if 'recession' in polymarket_contract.title.lower():
    signal = analyze_recession_probability(
        unemployment_var=kalshi_features['unemployment_variance'],
        gdp_mean=kalshi_features['gdp_mean']
    )
elif 'fed' in polymarket_contract.title.lower():
    signal = analyze_fed_decision(
        fed_rate_mean=kalshi_features['fed_rate_mean'],
        fed_rate_skew=kalshi_features['fed_rate_skewness']
    )

```

```
)

# Compare Polymarket price vs Kalshi-implied probability
polymarket_implied_prob = polymarket_contract.yes_price
kalshi_implied_prob = kalshi_features['event_probability']

if kalshi_implied_prob > polymarket_implied_prob + 0.05:
    return 'BUY_YES' # Kalshi suggests event more likely
elif kalshi_implied_prob < polymarket_implied_prob - 0.05:
    return 'BUY_NO'
return 'HOLD'
```

3.2 Volatility Skew from Asymmetric News Response

Definition: Volatility skew refers to the uneven distribution of uncertainty across different outcomes.

Kalshi Finding: Positive CPI surprises move Fed funds rate expectations ~**4x more** than negative surprises of equal magnitude[2].

Example:

CPI comes in 0.2% ABOVE expectations:

→ Fed rate mean increases by ~0.08% (8 bps)

CPI comes in 0.2% BELOW expectations:

→ Fed rate mean decreases by ~0.02% (2 bps)

Why Asymmetry Exists:

- Fed is more responsive to upside inflation risk (mandate to control inflation)
- Market prices in "inflation persistence" after positive shocks
- Downside surprises seen as temporary relief, not trend change

Measuring Volatility Skew:

```
def calculate_volatility_skew(kalshi_distribution_before,
kalshi_distribution_after,
cpi_surprise):
    """Measure how news changes distribution shape"""
```

```

# Calculate moments before and after
before_mean = kalshi_distribution_before.mean
before_var = kalshi_distribution_before.variance
before_skew = calculate_skewness(kalshi_distribution_before)

after_mean = kalshi_distribution_after.mean
after_var = kalshi_distribution_after.variance
after_skew = calculate_skewness(kalshi_distribution_after)

# Measure asymmetric response
if cpi_surprise > 0: # Positive shock
    response_type = 'POSITIVE_SHOCK'
    mean_shift = after_mean - before_mean # Expect large upward shift
    var_change = after_var - before_var # Expect variance decrease
    skew_change = after_skew - before_skew # Expect right tail compression
else: # Negative shock
    response_type = 'NEGATIVE_SHOCK'
    mean_shift = after_mean - before_mean # Expect small downward shift
    var_change = after_var - before_var
    skew_change = after_skew - before_skew # Expect left tail expansion

return {
    'response_type': response_type,
    'mean_shift': mean_shift,
    'volatility_change': np.sqrt(after_var) - np.sqrt(before_var),
    'skewness_change': skew_change,
    'asymmetry_ratio': abs(mean_shift) / abs(cpi_surprise) # Sensitivity
}

```

Trading Implications:

1. Directional Bias:

After positive CPI surprise → expect Fed rate increase

```
if cpi_surprise > 0:  
    # Market may underprice Fed hawkishness  
    position = 'BUY contracts for higher Fed rates'
```

After negative CPI surprise → modest Fed rate decrease

```
elif cpi_surprise < 0:  
    # Market may overprice Fed dovishness  
    position = 'SELL contracts for aggressive cuts'
```

2. Volatility Collapse Trade:

After ANY major news release, variance drops sharply as uncertainty resolves[2].

```
def trade_volatility_collapse(event_type='CPI_RELEASE'):  
    # BEFORE news release  
    kalshi_dist_before = fetch_distribution(event_type, time='pre-release')  
    volatility_before = np.sqrt(kalshi_dist_before.variance)
```

```
    # AFTER news release  
    kalshi_dist_after = fetch_distribution(event_type, time='post-release')  
    volatility_after = np.sqrt(kalshi_dist_after.variance)  
  
    volatility_collapse_pct = (volatility_before - volatility_after) / volatility_before  
  
    # Research finding: Volatility collapses most after ZERO-SURPRISE releases  
    if abs(actual_cpi - expected_cpi) < 0.05: # Zero surprise  
        expected_collapse = 0.60 # 60% variance reduction  
    else:  
        expected_collapse = 0.40 # 40% variance reduction
```



```
# Trading strategy
if volatility_collapse_pct > expected_collapse:
    # Volatility collapsed more than expected → range-bound ahead
    strategy = 'mean_reversion' # Fade extremes
else:
    # Volatility still elevated → trend may continue
    strategy = 'momentum' # Follow the move
```

3.3 Feature Engineering for Bots

Volatility Skew Features:

```
class KalshiVolatilityFeatures:
    def __init__(self, kalshi_api):
        self.api = kalshi_api
        self.history = [] # Store past distributions

    def extract_skew_features(self, event_id, lookback_days=30):
        """Generate features capturing asymmetric response patterns"""

        current_dist = self.api.get_distribution(event_id)

        features = {
            # Baseline moments
            'current_mean': current_dist.mean,
            'current_variance': current_dist.variance,
            'current_skewness': calculate_skewness(current_dist),

            # Tail risk measures
            'prob_upside_tail': sum(current_dist.probs[current_dist.outcomes > curr
            'prob_downside_tail': sum(current_dist.probs[current_dist.outcomes < c
            'tail_asymmetry': prob_upside_tail / (prob_downside_tail + 1e-6),

            # Time series of variance (volatility regime)
            'variance_ma_7d': np.mean([d.variance for d in self.history[-7:]]),
            'variance_ma_30d': np.mean([d.variance for d in self.history[-30:]]),
            'variance_trend': (variance_ma_7d - variance_ma_30d) / variance_ma_30d
```

```

# Skewness dynamics
'skewness_ma_7d': np.mean([calculate_skewness(d) for d in self.history])
'skewness_reversal': current_dist.skewness - skewness_ma_7d,

# Cross-moment features
'mean_variance_ratio': current_dist.mean / (current_dist.variance + 1e-6)

# Event-driven features (if news release occurred)
'post_news_variance_drop': self._calculate_post_news_variance_change(
'surprise_response_ratio': self._calculate_asymmetric_response()
}

# Update history
self.history.append(current_dist)
if len(self.history) > lookback_days:
    self.history.pop(0)

return features

def _calculate_post_news_variance_change(self):
    """Measure variance collapse after news"""
    if len(self.history) < 2:
        return 0

    # Compare variance before and after major news
    pre_news_var = self.history[-2].variance
    post_news_var = self.history[-1].variance

    return (pre_news_var - post_news_var) / pre_news_var

def _calculate_asymmetric_response(self):
    """Measure if positive vs negative shocks move mean differently"""
    if len(self.history) < 10:
        return 1.0 # Neutral

    positive_shocks = []
    negative_shocks = []

```

```

for i in range(1, len(self.history)):
    surprise = self.history[i].mean - self.history[i-1].mean
    if surprise > 0:
        positive_shocks.append(abs(surprise))
    elif surprise < 0:
        negative_shocks.append(abs(surprise))

avg_positive = np.mean(positive_shocks) if positive_shocks else 0
avg_negative = np.mean(negative_shocks) if negative_shocks else 0

# Ratio > 1 means positive shocks move mean more (typical for inflation/r
return avg_positive / (avg_negative + 1e-6)

```

3.4 Implementation Strategy

Complete Workflow:

```

def kalshi_volatility_trading_strategy(polymarket_contract):
    """Trade Polymarket using Kalshi volatility skew signals"""

```

```

# 1. Identify relevant Kalshi market
related_kalshi_event = map_polymarket_to_kalshi(polymarket_contract)

# 2. Extract features
kalshi_features = KalshiVolatilityFeatures(kalshi_api)
features = kalshi_features.extract_skew_features(related_kalshi_event)

# 3. Trading logic based on regime

# REGIME 1: Pre-News High Volatility
if features['variance_trend'] > 0.1 and days_until_news_release < 3:
    # Volatility is expanding before news → buy volatility
    # (In Polymarket: take positions expecting price movement)
    signal = 'INCREASE_POSITION_SIZE' # Expect movement

# REGIME 2: Post-News Volatility Collapse
elif features['post_news_variance_drop'] > 0.5:
    # Volatility collapsed > 50% → range-bound ahead

```

```

# (In Polymarket: mean reversion, fade extremes)
if polymarket_contract.yes_price > 0.7:
    signal = 'BUY_NO' # Overpriced
elif polymarket_contract.yes_price < 0.3:
    signal = 'BUY_YES' # Underpriced
else:
    signal = 'HOLD'

# REGIME 3: Asymmetric Response Detected
elif features['surprise_response_ratio'] > 2.0:
    # Positive shocks move 2x more than negative
    # If we expect positive shock → aggressive positioning
    expected_news_direction = predict_news_surprise()

    if expected_news_direction == 'POSITIVE':
        # Expect large move → position aggressively
        signal = 'BUY_YES' if features['tail_asymmetry'] > 1.5 else 'BUY_NO'
    else:
        # Negative shock has muted impact → modest positioning
        signal = 'SMALL_BUY_NO'

# REGIME 4: Normal Conditions
else:
    # Use standard Kalshi-Polymarket arbitrage
    kalshi_implied_prob = features['current_mean']
    polymarket_price = polymarket_contract.yes_price

    if kalshi_implied_prob > polymarket_price + 0.05:
        signal = 'BUY_YES'
    elif kalshi_implied_prob < polymarket_price - 0.05:
        signal = 'BUY_NO'
    else:
        signal = 'HOLD'

return signal

```

4. Integrated System Architecture

4.1 Combining All Strategies

Hierarchical Decision Framework:

LEVEL 1: TIPP-MADDPG (Portfolio Risk Management)

↓ Determines maximum position size

LEVEL 2: LLM Alpha Miner (Signal Generation)

↓ Generates trading signals from multimodal analysis

LEVEL 3: Kalshi Volatility Features (Regime Detection)

↓ Adjusts signal strength based on macro regime

LEVEL 4: Execution (Order Placement)

↓ Final trade execution with optimal pricing

Implementation:

```
class IntegratedPolymarketBot:
```

```
def init(self, initial_capital, config):
```

```
    self.capital = initial_capital
```

```
    self.config = config
```

```
        # Initialize components
        self.tipp_manager = TIPPPortfolioManager(
            floor_pct=config['floor_pct'],
            multiplier=config['multiplier']
        )

        self.maddpg_agents = [
            MADDPGAgent(agent_id=i, num_agents=config['num_agents'])
            for i in range(config['num_agents'])
        ]

        self.llm_alpha_miner = FinAgentAlphaMiner(
            market_intelligence=MarketIntelligenceModule(),
            reflection=DualLevelReflection(),
            memory=VectorMemoryDB()
```

```

)

self.kalshi_features = KalshiVolatilityFeatures(kalshi_api)

def execute_trading_cycle(self, current_date):
    """Main trading loop combining all strategies"""

    # STEP 1: Get available contracts
    contracts = polymarket_api.get_active_contracts()

    # STEP 2: TIPP Portfolio Constraint
    current_value = self.calculate_portfolio_value()
    floor = self.tipp_manager.calculate_floor(current_value)
    cushion = current_value - floor
    max_total_exposure = self.config['multiplier'] * cushion

    # STEP 3: For each contract, generate signals
    contract_signals = []

    for contract in contracts:
        # LLM Alpha Mining
        llm_signal = self.llm_alpha_miner.generate_signal(
            contract=contract,
            current_date=current_date
        )

        # Kalshi Volatility Features
        kalshi_features = self.kalshi_features.extract_skew_features(
            map_polymarket_to_kalshi(contract)
        )

        # Regime adjustment
        regime = self.detect_volatility_regime(kalshi_features)
        adjusted_confidence = self.adjust_confidence(
            llm_signal['confidence'],
            regime
        )

```

```

# MADDPG agents vote
state = self.construct_state(contract, llm_signal, kalshi_features)
agent_actions = [agent.select_action(state) for agent in self.maddpg_agen

# Aggregate signal
final_signal = self.aggregate_signals(
    llm_signal,
    agent_actions,
    adjusted_confidence
)

contract_signals.append({
    'contract': contract,
    'signal': final_signal,
    'confidence': adjusted_confidence
})

# STEP 4: Position sizing with TIPP constraint
positions = self.size_positions(
    contract_signals,
    max_total_exposure
)

# STEP 5: Execute trades
for position in positions:
    self.execute_trade(position)

# STEP 6: Update TIPP floor (ratchet mechanism)
new_value = self.calculate_portfolio_value()
self.tipp_manager.update_floor(new_value)

# STEP 7: Store experience and train agents
rewards = self.calculate_rewards()
for i, agent in enumerate(self.maddpg_agents):
    agent.store_transition(state, agent_actions[i], rewards[i], new_state)
    agent.train()

# STEP 8: Reflect and update memory

```

```

self.llm_alpha_miner.high_level_reflection(
    trading_history=self.get_recent_trades(),
    current_date=current_date
)

def detect_volatility_regime(self, kalshi_features):
    """Classify current market regime"""
    if kalshi_features['variance_trend'] > 0.1:
        return 'HIGH_VOLATILITY_EXPANDING'
    elif kalshi_features['post_news_variance_drop'] > 0.5:
        return 'LOW_VOLATILITY_COLLAPSED'
    elif kalshi_features['surprise_response_ratio'] > 2.0:
        return 'ASYMMETRIC_RESPONSE'
    else:
        return 'NORMAL'

def adjust_confidence(self, base_confidence, regime):
    """Adjust LLM confidence based on volatility regime"""
    adjustments = {
        'HIGH_VOLATILITY_EXPANDING': 0.8, # Reduce confidence
        'LOW_VOLATILITY_COLLAPSED': 1.2, # Increase (range-bound)
        'ASYMMETRIC_RESPONSE': 1.1,     # Slight increase
        'NORMAL': 1.0
    }
    return min(base_confidence * adjustments[regime], 100)

def size_positions(self, signals, max_exposure):
    """Kelly Criterion with TIPP constraint"""
    positions = []

    for signal_data in signals:
        contract = signal_data['contract']
        signal = signal_data['signal']
        confidence = signal_data['confidence'] / 100

        if signal == 'HOLD':
            continue

```



```

# Kelly Criterion:  $f^* = (p \cdot b - q) / b$ 
# p = confidence, q = 1-p, b = odds
price = contract.yes_price if signal == 'BUY_YES' else contract.no_price
odds = (1 - price) / price # Payout ratio

kelly_fraction = (confidence * odds - (1 - confidence)) / odds
kelly_fraction = max(0, min(kelly_fraction, 0.25)) # Cap at 25%

position_size = kelly_fraction * max_exposure

positions.append({
    'contract': contract,
    'action': signal,
    'size': position_size,
    'price': price
})

# Normalize if total exceeds max_exposure
total_exposure = sum(p['size'] for p in positions)
if total_exposure > max_exposure:
    scale_factor = max_exposure / total_exposure
    for p in positions:
        p['size'] *= scale_factor

return positions

```

4.2 Backtesting Framework

```

class PolymarketBacktester:
    def init(self, bot, historical_data, start_date, end_date):
        self.bot = bot
        self.data = historical_data
        self.start_date = start_date
        self.end_date = end_date

```

```

        self.portfolio_history = []
        self.trade_history = []

```

```

def run_backtest(self):
    """Simulate bot trading over historical period"""

    for date in pd.date_range(self.start_date, self.end_date):
        # Set data for this date
        self.set_market_state(date)

        # Bot makes decisions
        self.bot.execute_trading_cycle(date)

        # Record state
        self.portfolio_history.append({
            'date': date,
            'value': self.bot.calculate_portfolio_value(),
            'positions': self.bot.get_positions(),
            'floor': self.bot.tipp_manager.floor
        })

    # Calculate metrics
    returns = self.calculate_returns()
    sharpe = self.calculate_sharpe_ratio(returns)
    max_dd = self.calculate_max_drawdown()
    win_rate = self.calculate_win_rate()

    return {
        'total_return': returns[-1],
        'annualized_return': self.annualize(returns[-1]),
        'sharpe_ratio': sharpe,
        'max_drawdown': max_dd,
        'win_rate': win_rate,
        'trade_history': self.trade_history,
        'portfolio_history': self.portfolio_history
    }

def calculate_max_drawdown(self):
    """Maximum peak-to-trough decline"""
    values = [p['value'] for p in self.portfolio_history]
    peak = values[0]

```

```
max_dd = 0

for value in values:
    if value > peak:
        peak = value
    dd = (peak - value) / peak
    max_dd = max(max_dd, dd)

return max_dd
```

5. Production Deployment Considerations

5.1 Infrastructure Requirements

Computational Resources:

components:

llm_inference:

model: gpt-4-turbo

requests_per_day: ~500

cost_estimate: \$200-500/day

latency: 2-10 seconds

maddpg_training:

gpu: NVIDIA A100 (40GB)

training_frequency: daily

training_time: 2-4 hours

vector_database:

provider: Pinecone / Weaviate

dimensions: 1536 (OpenAI embeddings)

vectors_stored: 100,000+

data_feeds:

- polymarket_api: Real-time contract prices

- kalshi_api: Macro distributions

- news_api: Financial news (Bloomberg, Reuters)

- social_api: Twitter/Reddit sentiment

5.2 Risk Management

Hard Constraints:

```
risk_limits = {  
'max_position_pct': 0.25, # Max 25% in single contract  
'max_leverage': 1.0, # No leverage  
'max_correlation': 0.7, # Avoid over-concentration  
'min_liquidity': 10000, # Minimum $10k volume  
'max_drawdown_stop': 0.20 # Stop trading if -20% from peak  
}
```

Monitoring:

```
def risk_monitoring(bot):  
    """Real-time risk checks"""
```

```
# Check TIPP floor breach  
if bot.portfolio_value < bot.tipp_manager.floor:  
    alert('FLOOR BREACHED - Emergency liquidation')  
    bot.liquidate_all_positions()  
  
# Check correlation limits  
positions = bot.get_positions()  
correlation_matrix = calculate_correlations(positions)  
if correlation_matrix.max() > risk_limits['max_correlation']:  
    alert('HIGH CORRELATION - Reduce correlated positions')  
  
# Check liquidity  
for position in positions:  
    if position.daily_volume < risk_limits['min_liquidity']:  
        alert(f'LOW LIQUIDITY - {position.contract_id}')
```

5.3 Performance Optimization

Latency Reduction:

Cache LLM responses for similar queries

```
@lru_cache(maxsize=1000)
def cached_llm_call(prompt_hash):
    return llm_call(prompt)
```

Batch processing

```
def batch_market_intelligence(contracts):
    """Process multiple contracts in parallel"""
    with ThreadPoolExecutor(max_workers=10) as executor:
        futures = [executor.submit(process_contract, c) for c in contracts]
        results = [f.result() for f in futures]
    return results
```

6. Future Research Directions

6.1 Advanced Techniques

1. Hierarchical Reinforcement Learning:

- High-level policy selects strategy (momentum/mean-reversion/volatility)
- Low-level policy executes within selected strategy
- Faster adaptation to regime changes

2. Transformer-Based Market Models:

- Replace MADDPG with Transformer architecture
- Attention mechanism to identify relevant features
- Better handling of long-range dependencies

3. Causal Inference:

- Move beyond correlation to causation
- Identify true drivers of price movements
- Reduce spurious signals

4. Multi-Task Learning:

- Train single model across multiple prediction markets
- Share representations between related contracts
- Improve sample efficiency

6.2 Open Questions

1. **Risk Premia in Prediction Markets:** Are Kalshi/Polymarket prices truly unbiased? What systematic biases exist?
2. **Optimal Reflection Frequency:** How often should LLM reflect on decisions? Daily? Weekly?
3. **TIPP Parameter Tuning:** Optimal floor percentage and multiplier for prediction markets?
4. **Multi-Agent Coordination:** How many agents in MADDPG is optimal? Diversity vs complexity tradeoff?

Conclusion

This report has presented a comprehensive framework for algorithmic trading in prediction markets, synthesizing insights from portfolio insurance theory, multi-agent reinforcement learning, large language model reasoning, and macroeconomic volatility analysis. The strategies outlined—TIPP-MADDPG for risk management, FinAgent-style LLM alpha mining for signal generation, and Kalshi volatility features for regime detection—represent the frontier of quantitative trading research applied to the emerging prediction market ecosystem.

Key takeaways for implementation:

1. **TIPP-MADDPG** provides robust downside protection while allowing adaptive position sizing through multi-agent learning
2. **LLM Alpha Miners** excel at processing multimodal data and learning from historical patterns through reflection
3. **Kalshi Volatility Skew** offers unique macro features unavailable in traditional markets, particularly for detecting asymmetric news responses
4. **Integration** of all three approaches creates a comprehensive system balancing return generation with risk management

The research demonstrates that prediction markets, while nascent, offer exploitable inefficiencies through sophisticated quantitative methods. As these markets mature and liquidity deepens, the strategies outlined here will become increasingly viable for production deployment.

References

- [1] Zhang, H., Shi, Z., Hu, Y., Ding, W., Kuruoğlu, E. E., & Zhang, X. P. (2023). Optimizing trading strategies in quantitative markets using multi-agent reinforcement learning. *arXiv preprint arXiv:2303.11959*.
- [2] Diercks, A. M., Katz, J. D., & Wright, J. H. (2026). Kalshi and the rise of macro markets. *Finance and Economics Discussion Series 2026-010*. Washington: Board of Governors of the Federal Reserve System.
<https://doi.org/10.17016/FEDS.2026.010>
- [3] Zhang, W., Zhao, L., Xia, H., Sun, S., Sun, J., Qin, M., Li, X., Zhao, Y., Zhao, Y., Cai, X., Zheng, L., Wang, X., & An, B. (2024). A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*, Barcelona, Spain.
- [4] Core AXA IM. (2024). Understanding portfolio insurance management (CPPI/TIPP). *Investment Insights*.
- [5] QuantPedia. (2023). Time invariant portfolio protection. *QuantPedia Research*.
- [6] PMC. (2021). Investors' perspective on portfolio insurance. *NCBI PMC8370861*.
- [7] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. In *31st Conference on Neural Information Processing Systems (NeurIPS 2017)*.
- [8] Dozendiamonds. (2025). Volatility regime shifting: How to detect market shifts early. *Trading Research*.
- [9] Tradefundrr. (2025). Event-driven trading strategies: Master market events efficiently. *Trading Education*.

[10] Binance. (2026). Review of prediction market arbitrage strategies. *Binance Square*.